Southwest WI Technical College

10-152-117 Python Programming

Course Design

Course Information

Alternate Title	Foundations of Programming for AI-Augmented Engineers
Description	Learners explore Python programming by designing and implementing solutions to real-world problems. Students develop foundational programming skills using variables, data types, conditionals, loops, functions, and data structures. The course emphasizes problem-solving, debugging, and code comprehension while introducing the use of AI-assisted tools to support development. Learners practice building, evaluating, and refining programs, both independently and with AI assistance. By the end of the course, students are able to create, understand, and explain basic Python programs and apply structured thinking to solve problems.
Career Cluster	Information Technology
Instructional Level	A.A.S. - Associate in Applied Science
Total Credits	2
Total Hours	54

Types of Instruction

Instruction Type	Credits/Hours
(A) Lecture	18
(B) Lab	36

Textbooks

Learn Python Programming: A comprehensive, up-to-date, and definitive guide to learning Python - 4th ed. Edition - Fabrizio Romano/Heinrich Kruger

The Missing Tools: How Developers Learn to Think Quickly - 1st Edition - Cash Myers

Python Programming Language: a QuickStudy Laminated Reference Guide Pamphlet – Robin Nixon – (Optional)

Course Competencies

1. Analyze problems and define program logic

Assessment Strategies

1.1. Project assignments and quiz

Criteria

1.1. Break problems into step-by-step logic
1.2. Identify inputs, outputs, and constraints
1.3. Represent solutions using structured logic (pseudocode or code)
1.4. Distinguish between clear and ambiguous problem statements

Learning Objectives

1.a. Define program logic and problem decomposition
1.b. Identify inputs, outputs, and constraints
1.c. Describe step-by-step problem-solving approaches

2. Implement Python programs using core programming constructs

Assessment Strategies

2.1. Project assignments and quiz

Criteria

2.1. Write Python programs using variables, data types, conditionals, and loops
2.2. Use lists and dictionaries to manage data
2.3. Define and use functions
2.4. Apply basic file input/output operations
2.5. Produce readable and structured code

Learning Objectives

2.a. Identify Python syntax and constructs
2.b. Explain variables, data types, and control structures
2.c. Describe functions and basic data structures

3. Utilize AI-assisted tools to develop and refine Python programs

Assessment Strategies

3.1. Project assignments and quiz

Criteria

3.1. Use AI tools to generate or assist in writing code
3.2. Provide clear prompts that describe the intended solution
3.3. Modify and adapt AI-generated code to meet requirements
3.4. Integrate AI-generated components into working programs

Learning Objectives

3.a. Describe how AI tools support programming
3.b. Identify effective prompting strategies
3.c. Explain how to adapt generated code

4. Evaluate and debug Python programs for correctness and clarity

Assessment Strategies

4.1. Project assignments and quiz

Criteria

4.1. Identify and fix syntax and logical errors
4.2. Test programs using sample inputs
4.3. Explain how a program works step-by-step
4.4. Compare different approaches to solving the same problem

Learning Objectives

4.a. Define debugging and testing
4.b. Identify common programming errors
4.c. Explain program execution flow

5. Adapt programming approaches with and without AI assistance

Assessment Strategies

5.1. Project assignments and quiz

Criteria

5.1. Write basic programs without relying on AI tools
5.2. Use AI to improve or extend existing solutions
5.3. Explain their code to others clearly
5.4. Adjust solutions based on constraints or requirements

Learning Objectives

5.a. Describe differences between manual and AI-assisted development
5.b. Explain when AI use is appropriate
5.c. Identify strategies for adapting solutions